

Architecture Numérique et Modélisation Physiologique : Ingénierie d'une Application de Musculation Orientée Données sous Android

L'industrie du conditionnement physique numérique traverse une période de mutation technologique sans précédent. L'époque où les pratiquants de la musculation se contentaient de carnets d'entraînement physiques ou de simples applications de prise de notes est révolue. Aujourd'hui, la demande s'oriente massivement vers des écosystèmes logiciels hautement analytiques, capables non seulement d'enregistrer des charges et des répétitions, mais de modéliser l'hypertrophie musculaire, de quantifier la fatigue systémique et d'optimiser la planification périodisée de l'entraînement. La création d'une application Android de pointe, s'inspirant des leaders du marché tels que Hevy, Strong ou FitNotes, exige une convergence absolue entre les sciences du sport (cinésiologie, physiologie de l'hypertrophie) et l'ingénierie logicielle moderne (architecture "Offline-First", bases de données relationnelles embarquées, interfaces réactives via Jetpack Compose).

Ce rapport exhaustif détaille les spécifications fonctionnelles, les algorithmes mathématiques, les paradigmes d'interface utilisateur et les fondations architecturales nécessaires au développement d'une plateforme de musculation de nouvelle génération. Chaque dimension du cycle de développement est analysée, de la modélisation complexe du volume d'entraînement par groupe musculaire à l'intégration des interfaces de programmation (API) matérielles telles que Health Connect et Wear OS.

Analyse Comparative et Paradigmes de l'Écosystème Concurrentiel

Pour concevoir une application surpassant les standards actuels, il est impératif de déconstruire les philosophies de conception adoptées par les acteurs dominants du marché. Le paysage concurrentiel de 2026 se divise en plusieurs catégories distinctes, allant de l'outil ultra-minimaliste à la plateforme sociale gamifiée, en passant par l'assistant propulsé par l'intelligence artificielle.

L'Approche Sociale et Visuelle : Le Cas d'Étude Hevy

L'application Hevy s'est imposée comme une référence incontournable en fusionnant le suivi analytique rigoureux avec des mécaniques d'engagement issues des réseaux sociaux.¹

L'interface de Hevy est conçue pour transformer des données brutes en visualisations hautement digests. La fonctionnalité phare réside dans son graphique corporel dynamique ("Body Graph"), qui met en surbrillance les groupes musculaires sollicités au cours des sept

derniers jours, offrant ainsi à l'utilisateur une vision instantanée de sa répartition volumique.¹ Les statistiques avancées proposent une décomposition granulaire du volume d'entraînement total par muscle, accompagnée de graphiques circulaires illustrant la distribution musculaire sur des échelles de temps paramétrables.¹ Il est crucial de noter la stratégie de monétisation associée : la version gratuite restreint l'accès aux données historiques aux trois derniers mois, tandis que l'abonnement "Pro" (tarifié à environ 59,99 \$ annuellement suite aux augmentations tarifaires de 2026) débloque l'intégralité de l'historique et des filtres temporels.¹ Au-delà de la visualisation, l'architecture de rétention de Hevy repose sur son flux d'actualités ("Social Feed").⁵ Ce flux affiche les séances d'entraînement des athlètes suivis, incluant le volume total, les records personnels battus, et la fréquence cardiaque moyenne si la séance a été enregistrée via une montre connectée.⁵ La viralité de l'application est ingénieusement stimulée par des fonctionnalités de partage de routines et de dossiers d'entraînement. Les utilisateurs peuvent générer des liens ou des images récapitulatives de leurs programmes pour les partager sur des forums externes, permettant à d'autres utilisateurs d'importer et de cloner ces routines directement dans leur propre profil en un seul clic.⁶

L'Approche Structurée et Analytique : Le Modèle Strong

Strong se positionne sur un segment légèrement différent, ciblant historiquement une démographie d'haltérophiles et de culturistes plus avancés, privilégiant la structure et la performance pure au détriment des interactions sociales.⁹ La supériorité de Strong réside dans sa gestion de la complexité programmatique. L'application gère avec une fluidité exceptionnelle la création de modèles (templates) complexes impliquant des circuits, des supersets, et un historique de modification granulaire.⁹

Pour les utilisateurs employant des routines hautement personnalisées telles que des divisions "Push/Pull/Legs" (PPL), le flux de travail de Strong minimise considérablement les frictions lors de la saisie.⁹ De plus, Strong intègre des fonctionnalités très demandées par les utilisateurs expérimentés, telles que la possibilité d'épingler des notes persistantes sur des exercices spécifiques, une fonction souvent négligée par ses concurrents axés sur les réseaux sociaux.¹¹ L'application propose également des exportations CSV complètes, permettant aux analystes de données amateurs d'exporter leurs métriques vers des tableurs externes pour une modélisation personnalisée.⁹ Sur le plan tarifaire, Strong maintient une position compétitive avec un abonnement premium annuel autour de 29,99 \$ et une option d'achat à vie, tout en ne limitant pas le nombre d'exercices personnalisés dans sa version gratuite.⁴

Le Minimalisme et la Souveraineté des Données : FitNotes

À l'opposé du spectre de la complexification logicielle se trouve FitNotes, une application célébrée pour son minimalisme absolu et son respect de la propriété des données.¹² Conçue pour être extrêmement légère (pesant moins de 5 mégaoctets), FitNotes rejette les fonctionnalités sociales, la gamification et les modèles d'abonnement.¹² L'application se concentre exclusivement sur l'enregistrement des séances dans une interface épurée permettant une navigation rapide par glissement (swipe) entre les jours du calendrier.¹²

L'atout majeur de FitNotes réside dans son architecture entièrement hors ligne ("offline"), ce qui s'avère particulièrement pertinent pour les utilisateurs s'entraînant dans des installations sportives dépourvues de couverture réseau.¹³ L'absence totale de fenêtres contextuelles incitatives ("pop-ups") ou de publicités garantit une expérience exempte de surcharge cognitive, un paradigme de conception essentiel pour les athlètes souhaitant maintenir une concentration ininterrompue durant leur séance.¹³

L'Émergence de la Programmation Algorithmique : Fitbod et SensAI

Le marché évolue rapidement vers la génération algorithmique de programmes. Des applications comme Fitbod génèrent dynamiquement des séances d'entraînement en s'adaptant à l'équipement disponible dans la salle de sport de l'utilisateur et à son état de fatigue musculaire modélisé.⁴ Plus récemment, des plateformes comme SensAI intègrent des modèles de langage (LLM) et exploitent les données biométriques issues des vêtements connectés (variabilité de la fréquence cardiaque, qualité du sommeil, récupération systémique) pour adapter les prescriptions d'entraînement en temps réel de manière conversationnelle.⁴ Bien que la création d'une intelligence artificielle générative dépasse la simple création d'un journal d'entraînement, la conception de la base de données doit anticiper l'intégration future de telles capacités analytiques.

Synthèse Concurrentielle des Fonctionnalités Clés

L'analyse comparative des leaders du marché permet de dégager les vecteurs de différenciation stratégique. Le tableau suivant synthétise les paradigmes opérationnels des principales applications étudiées.

Application	Philosophie Principale	Gestion du Volume Musculaire	Connectivité et Wearables	Modèle Économique (Tarifs 2026)
Hevy	Communauté, Suivi visuel, Motivation	Graphiques corporels, Séries par muscle/semaine	Logging complet (Apple Watch / Wear OS)	Freemium (~59,99 \$/an)
Strong	Performance, Haltérophilie structurée	Exportation CSV, Analyse externe détaillée	Focus sur l'exécution hors-ligne (Apple Watch)	Freemium (~29,99 \$/an ou Achat à vie)
FitNotes	Minimalisme, Vitesse, Hors-ligne	Historique basique par exercice	Support rudimentaire	100% Gratuit (Sans publicité)
Fitbod	Génération algorithmique, Adaptation	Calcul automatisé de la fatigue locale	Suivi d'activité standard	Premium (~95,99 \$/an)
SensAI	Coaching IA, Données biométriques	Adaptation via Variabilité de la Fréquence	Intégration profonde (Sommeil,	Freemium (Modèle hybride)

		Cardiaque	Récupération)	
--	--	-----------	---------------	--

Le Cœur Analytique : Modélisation Mathématique de l'Hypertrophie

La proposition de valeur fondamentale d'une application centrée sur les statistiques réside dans sa capacité à traiter des vecteurs de données brutes (charges, répétitions, temps de repos) pour en dériver des métriques physiologiques exploitables. Le suivi rigoureux du volume hebdomadaire par groupe musculaire constitue le paradigme dominant de la science de l'hypertrophie contemporaine.³

Quantification du Volume : L'Algorithme d'Attribution Primaire et Secondaire

L'amélioration logique par rapport aux anciens modèles de répartition par parties du corps ("bro splits") consiste à comptabiliser le nombre total de séries effectives pour chaque groupe musculaire par semaine.¹⁵ L'application doit agréger les données des sessions validées et attribuer mathématiquement chaque série à des cibles musculaires spécifiques.³ Cependant, une attribution binaire simple (où un exercice cible un seul muscle) est physiologiquement erronée en raison de la nature polyarticulaire de la majorité des mouvements de musculation. Pour qu'un tableau de bord statistique reflète fidèlement le stimulus hypertrophique, il est impératif d'appliquer un coefficient de pondération aux muscles impliqués. L'approche standard, validée par des applications spécialisées comme MacroFactor ou Hevy, utilise une modélisation fractionnaire.¹⁶ Dans ce modèle algorithmique, un muscle agissant comme moteur principal (Muscle Primaire) se voit attribuer une valeur de 1.0 série. Un muscle agissant comme synergiste ou stabilisateur dynamique (Muscle Secondaire) se voit attribuer une valeur de 0.5 série.¹⁶

Prenons l'exemple d'une séance ciblant la chaîne postérieure. Si un athlète exécute un tirage vertical (Lat Pulldown), le grand dorsal agit comme muscle primaire (1.0), tandis que les biceps brachiaux interviennent significativement en tant que fléchisseurs du coude (muscle secondaire, 0.5).¹⁶ Si l'athlète réalise 10 séries d'exercices de tirage polyarticulaires pour le dos, l'algorithme doit automatiquement incrémenter le compteur de volume des biceps de 5 séries effectives.¹⁶

Néanmoins, la logique applicative doit intégrer des exceptions anatomiques cruciales. Des exercices tels que le soulevé de terre (Deadlift), le tirage à crémaillère (Rack Pull) ou le pull-over sollicitent le dos intensément mais n'impliquent aucune flexion active du coude.¹⁶ Par conséquent, la base de données relationnelle des exercices doit être méticuleusement modélisée : l'algorithme ne doit attribuer aucune fraction de série aux biceps lors de ces mouvements spécifiques, sous peine de fausser l'analyse de la récupération de l'utilisateur.¹⁶

Une attention méticuleuse doit également être portée aux mouvements polyarticulaires du bas du corps, où les muscles fessiers (glutes) accumulent rapidement un volume secondaire important lors des flexions sur jambes (squats) ou des presses à cuisses.¹⁶

Implémentation des Repères de Volume (Renaissance Periodization)

Pour transcender la simple fonction de journalisation et offrir une véritable valeur de coaching analytique, l'application doit intégrer des cadres conceptuels avancés, tels que les "Volume Landmarks" popularisés par le Dr Mike Israetel et la Renaissance Periodization (RP).¹⁹ Cette intégration permet à l'application d'évaluer si le volume comptabilisé est insuffisant, optimal ou excessif.

L'algorithme de modélisation doit évaluer le volume hebdomadaire agrégé de l'utilisateur à l'aune de quatre repères fondamentaux²⁰ :

1. **MV (Maintenance Volume)** : Le seuil quantitatif minimal requis pour préserver l'hypertrophie musculaire existante.²⁰
2. **MEV (Minimum Effective Volume)** : Le volume minimum déclenchant une cascade adaptative entraînant la croissance musculaire. Tout volume inférieur à ce seuil n'assure que la maintenance.²⁰
3. **MAV (Maximum Adaptive Volume)** : La fourchette volumique optimale (généralement évolutive au cours d'un mésocycle) au sein de laquelle les adaptations physiologiques sont maximisées.²⁰
4. **MRV (Maximum Recoverable Volume)** : La limite systémique et locale au-delà de laquelle l'organisme est incapable de dissiper la fatigue accumulée, menant inévitablement à des rendements décroissants et au surentraînement.²⁰

La base de données de l'application doit contenir des valeurs de référence standardisées pour chaque groupe musculaire, ajustables selon le profil de l'utilisateur. Le tableau suivant présente un extrait de ces valeurs de référence qui doivent être encodées dans la logique de l'application.

Groupe Musculaire	MV Recommandé (Séries/Semaine)	Fréquence d'Entraînement Recommandée
Dos (Back)	6	2 à 4 fois par semaine
Quadriceps	6	2 à 3 fois par semaine
Ischio-jambiers (Hamstrings)	3	2 à 3 fois par semaine
Fessiers (Glutes)	0	1 à 3 fois par semaine
Biceps	4	2 à 3 fois par semaine
Triceps	4	2 à 6 fois par semaine
Mollets (Calves)	0	2 à 6 fois par semaine
Deltoïdes Postérieurs	0	2 à 5 fois par semaine

Note : Les données proviennent des recommandations de la Renaissance Periodization.²¹

Pour rendre l'application dynamique, un algorithme de progression hebdomadaire doit être implémenté. L'application peut requérir de l'utilisateur une évaluation subjective de ses courbatures (DOMS) avant une nouvelle séance, et attribuer une pondération numérique pour ajuster la prescription de volume¹⁹ :

- L'absence totale de courbatures génère un score de 1 point, indiquant que le système

peut tolérer une augmentation du volume vers le MAV supérieur.¹⁹

- Une récupération adéquate bien avant la session génère 2 points, validant le maintien d'une progression volumique modérée.¹⁹
- Une récupération "juste à temps" génère 3 points, signalant que le volume actuel frôle le MRV.¹⁹
- La persistance de courbatures invalidantes génère 4 points, déclenchant une alerte algorithmique recommandant une phase de décharge (deload) ou une réduction drastique du volume pour la semaine en cours.¹⁹

Quantification de la Proximité de l'Échec : RPE et RIR

Le décompte des séries est dénué de sens si l'intensité de ces séries n'est pas quantifiée. Une série effectuée sans effort ne génère aucune tension mécanique significative et ne devrait pas être intégrée au calcul du volume effectif. Par conséquent, l'application doit implémenter un suivi rigoureux du RPE (Rate of Perceived Exertion) et du RIR (Reps in Reserve).²³

Le RPE s'articule sur une échelle subjective (généralement de 1 à 10), où un score de 10 correspond à un effort maximal absolu, interdisant l'exécution d'une répétition supplémentaire.²⁵

Le RIR, plus concret pour les exercices de musculation, quantifie directement le nombre de répétitions que l'athlète estime avoir "en réserve" avant d'atteindre l'échec musculaire (un RIR de 0 équivalant à un RPE de 10).²⁴ L'architecture logicielle doit permettre d'activer ou de désactiver ces champs de saisie de manière globale, ou spécifiquement pour certains exercices (par exemple, activer le RPE pour le squat lourd, mais le désactiver pour les exercices d'isolation mineurs).²³ L'interface utilisateur doit permettre la saisie rapide de ces métriques via des menus déroulants ou des sélecteurs numériques.

L'application peut également fournir des directives pédagogiques intégrées. Les utilisateurs surestiment souvent leur proximité de l'échec. Un module d'aide pourrait suggérer des protocoles de calibrage, tels que l'enregistrement vidéo des séries pour analyser la vitesse de la barre, ou l'exécution occasionnelle de blocs d'entraînement à faible volume visant l'échec momentané absolu pour recalibrer la perception sensorielle d'un véritable RIR 0.²⁴

Le Calculateur de Disques (Plate Calculator) : Une Nécessité Ergonomique

La fatigue neurologique induite par un entraînement lourd diminue considérablement les capacités de calcul mental. L'intégration d'un "Plate Calculator" est une exigence absolue pour toute application ciblant les haltérophiles.²⁷ Cet outil élimine la "gymnastique mentale" requise pour charger une barre olympique.³¹

L'implémentation logicielle d'un calculateur de disques repose sur un algorithme glouton (Greedy Algorithm) simple mais robuste. Les paramètres d'entrée comprennent le poids cible désiré par l'utilisateur, le poids de la barre à vide (paramétrable, par défaut 20 kg ou 45 lbs), et un inventaire de l'équipement disponible (le nombre et le poids de chaque disque possédé par l'utilisateur).²⁷

La logique s'articule comme suit : l'algorithme soustrait le poids de la barre au poids cible total.

Le résultat représente la masse totale des disques à charger. Ce chiffre est divisé par deux pour obtenir la masse à charger sur un seul côté de la barre.²⁹ L'algorithme glouton itère ensuite sur le tableau des disques disponibles, classés par ordre décroissant de poids. Il sélectionne le disque le plus lourd possible, le soustrait du poids restant à charger par côté, et répète l'opération itérativement jusqu'à ce que le poids restant atteigne zéro.²⁹ Le résultat est ensuite renvoyé à la couche de présentation de l'application (UI) pour générer une représentation visuelle claire des disques à empiler.²⁹ L'application doit également gérer les unités de mesure de manière fluide, permettant une conversion instantanée entre kilogrammes et livres, voire le microchargement (microloading) avec des disques fractionnaires.²⁸

Architecture Logicielle et Gestion des Données : Le Paradigme "Offline-First"

Les salles de sport, souvent situées dans des environnements souterrains ou des structures métalliques denses, souffrent intrinsèquement d'une mauvaise couverture réseau.¹³ Exiger une connexion Internet constante pour valider une série d'entraînement constitue une erreur de conception fatale, générant des temps de latence qui détruiront l'expérience utilisateur.³² L'architecture fondamentale de l'application doit donc reposer sur un paradigme "Offline-First" absolu.³³

Modélisation de la Base de Données Relationnelle (Room et SQLite)

Dans une architecture "Offline-First" sur Android, la source unique de vérité (Single Source of Truth) de l'application n'est pas le serveur cloud, mais la base de données locale hébergée sur le périphérique.³⁵ La bibliothèque Jetpack Room offre une couche d'abstraction puissante au-dessus de SQLite pour gérer cette persistance avec sécurité et robustesse.³³

Le schéma de la base de données doit modéliser fidèlement la complexité d'un programme d'entraînement. La difficulté principale réside dans la gestion des relations de type "plusieurs-à-plusieurs" (Many-to-Many). Un utilisateur peut créer plusieurs routines, et un exercice spécifique (par exemple, le "Développé Couché") peut être intégré dans de multiples routines distinctes.³² De plus, l'historique des performances nécessite une liaison entre une session d'entraînement précise, un exercice, et les multiples séries associées.⁴¹

La structure relationnelle exige la mise en place de tables de jonction (Bridge Tables) pour normaliser les données et éviter la duplication de l'inventaire des exercices.⁴⁰ Le schéma architectural recommandé se décompose ainsi :

Entité (Table SQL)	Rôle dans l'Application	Attributs Clés (Colonnes)	Relations
User	Stockage des données biométriques	user_id, weight, height	Possède de nombreux Workouts ⁴¹
Exercise	Le catalogue global (Dictionnaire)	exercise_id, name, primary_muscle,	Référencé mondialement ⁴⁰

		secondary_muscle	
Routine	Les modèles de programmes sauvegardés	routine_id, name, notes	Contient des exercices ⁴²
Routine_Exercise	Table de jonction (Many-to-Many)	routine_id, exercise_id, order_index	Résout le conflit Routine/Exercise ⁴⁰
Workout	La session réelle exécutée	workout_id, date, duration, volume	Contient de multiples séries ⁴¹
Workout_Exercise	L'exercice au sein d'une session précise	workout_exercise_id, workout_id, exercise_id, superset_id	Gère l'ordre et les enchaînements
Set (Série)	Les métriques quantitatives de l'effort	set_id, workout_exercise_id, weight, reps, rpe, type	Associé à un Workout_Exercise précis

Une attention particulière doit être portée à l'attribut `superset_id` dans l'entité `Workout_Exercise`. La gestion des supersets (l'exécution consécutive de deux exercices différents sans période de repos) nécessite un identifiant de regroupement logique dans la base de données. Tous les exercices partageant le même `superset_id` au sein d'une session donnée seront rendus visuellement par l'interface utilisateur comme un bloc d'entraînement unifié, permettant à l'utilisateur de lier leurs chronomètres de repos respectifs.³²

Synchronisation Asynchrone et File d'Attente

Une fois les données écrites localement dans la base Room avec un temps de latence quasi nul pour l'utilisateur, l'application doit gérer la synchronisation avec le serveur backend (pour la sauvegarde dans le cloud et la distribution dans le flux social) de manière transparente. Les appels au serveur doivent être extraits du fil d'exécution principal de l'interface graphique (Main Thread) pour éviter tout blocage (Application Not Responding - ANR).³⁶

Dans un flux de travail "Offline-First", les requêtes de lecture et d'écriture réseau sont mises en file d'attente.³⁴ Sur Android, cette file d'attente est implémentée via l'API `WorkManager`.³⁴ Le composant `WorkManager` permet de planifier des tâches différables avec des contraintes spécifiques. Par exemple, l'application peut ordonner à `WorkManager` de synchroniser l'historique d'entraînement uniquement lorsque l'appareil détecte une connexion réseau non facturée (Wi-Fi) et que la batterie est suffisamment chargée.⁴⁴ Cette approche garantit la résilience des données, même si l'application est fermée brutalement par le système d'exploitation.⁴⁶

Architecture Globale : MVVM et Flux de Données Unidirectionnel (UDF)

Pour garantir la maintenabilité d'un code source gérant des calculs statistiques complexes, il est essentiel d'adopter une architecture logicielle multicouche propre, telle que la "Clean

Architecture" combinée au motif Model-View-ViewModel (MVVM).³⁷

- **La Couche de Présentation (UI)** : Gère l'affichage via Jetpack Compose.
- **Les ViewModels** : Agissent comme intermédiaires, conservant l'état de l'interface (UI State) et réagissant aux événements déclenchés par l'utilisateur.³⁷
- **La Couche Domaine** : Contient les cas d'utilisation (Use Cases) encapsulant la logique métier pure, telle que le calcul du volume musculaire ou l'algorithme du Plate Calculator. Cette couche, isolée des dépendances Android spécifiques, facilite la création de tests unitaires rigoureux.³⁷
- **La Couche Données** : Constitue l'interface avec la base Room et les services réseau, orchestrant le principe de la source de vérité unique.³⁸

L'architecture s'appuie massivement sur la programmation réactive via Kotlin Coroutines et Flow. Les composants de l'interface utilisateur observent les flux de données (Flows) émis par la base de données Room. Toute insertion d'une nouvelle série déclenche automatiquement une mise à jour des flux, provoquant une recomposition instantanée de l'interface graphique sans nécessiter de rechargements de pages coûteux, implémentant ainsi un Flux de Données Unidirectionnel (UDF) élégant et performant.³²

Optimisation de l'Interface Utilisateur (UI) et Expérience Utilisateur (UX)

L'environnement de la salle de sport impose des contraintes ergonomiques extrêmes. Les utilisateurs manipulent leur smartphone avec des mains parfois moites, sous l'emprise de la fatigue centrale, avec des fenêtres de repos limitées à quelques dizaines de secondes. La charge cognitive requise pour interagir avec l'application doit s'approcher de zéro.

Le Défi de l'Onboarding et de la Rétention

L'accueil des nouveaux utilisateurs (Onboarding) détermine le taux d'adoption initial. L'erreur la plus fréquente, identifiée dans les études de cas ergonomiques sur les applications de santé, consiste à imposer un interrogatoire exhaustif (âge, poids, taille, objectifs) avant même de présenter la valeur fondamentale du produit.⁴⁷

Les principes d'UX exigent une simplification drastique. Il est recommandé de réduire l'onboarding à cinq étapes ou moins, et d'offrir systématiquement une option pour ignorer le processus et accéder immédiatement au tableau de bord.⁴⁸ L'objectif est de délivrer le "moment Aha" (la compréhension de la valeur de l'application) le plus rapidement possible, par exemple en permettant l'enregistrement immédiat d'une séance factice.⁴⁸ La typographie et les contrastes de couleurs doivent être particulièrement soignés ; les textes en rouge sur fond sombre, souvent utilisés pour signaler des erreurs, posent des problèmes d'accessibilité cruciaux pour les utilisateurs malvoyants et doivent être maniés avec précaution.⁴⁸

Jetpack Compose et Material 3 Expressive

Le développement natif Android moderne exige l'abandon des anciens fichiers XML au profit de Jetpack Compose, un framework déclaratif permettant de construire l'interface utilisateur

programmatically en Kotlin.³⁷ L'application doit adhérer aux normes du "Material Design 3" (M3), et plus particulièrement à sa récente évolution "Expressive", qui introduit des animations fluides et des composants morphologiques.⁵¹

L'architecture de navigation doit rester superficielle. L'utilisation d'une barre de navigation inférieure persistante (Bottom Navigation Bar) comprenant des onglets clairs (Accueil, Entraînement, Profil/Statistiques) permet aux utilisateurs de s'orienter instantanément, évitant l'enfouissement des fonctionnalités clés dans des menus tiroirs profonds, un défaut souvent reproché aux anciennes interfaces.⁵⁰ Pour gérer les différentes options au sein d'une vue (par exemple, filtrer les statistiques par "Semaine", "Mois" ou "Année"), l'utilisation des boutons segmentés (Segmented Buttons) du Material Design 3 est idéale.⁵³

Accélération de la Saisie des Données : Number Pickers et Assistances Audio

Le goulot d'étranglement majeur de l'UX d'une application de musculation est la saisie des poids et des répétitions. Les champs de texte standards (TextField) sollicitent l'affichage du clavier virtuel du système d'exploitation. L'apparition répétée de ce clavier obstrue l'écran, requiert de multiples frappes pour effacer et saisir de nouveaux chiffres, et induit une frustration considérable.⁵⁷

La solution logicielle optimale consiste à implémenter des sélecteurs numériques (NumberPicker) natifs via Jetpack Compose.⁵⁸ Ces composants permettent de définir un cadran rotatif ou des boutons d'incrément (+/-) superposés directement dans la cellule de l'exercice. L'utilisateur peut ainsi ajuster la charge ou le nombre de répétitions par de simples glissements verticaux ou tapotements, sans jamais quitter le contexte visuel de sa séance d'entraînement.⁵⁸

L'application doit également intégrer des heuristiques prédictives. Lorsqu'une nouvelle série est ajoutée, l'application (à l'image des logiques implémentées par Hevy ou FitNotes) doit automatiquement pré-remplir les champs avec les valeurs de la série précédente, ou mieux, avec les valeurs de la même séance exécutée la semaine passée.⁹ Cette duplication intelligente transforme l'acte de saisie en une simple validation, accélérant drastiquement le processus. Pour repousser les limites de la friction d'entrée, des études de conception émergentes explorent l'intégration d'assistants vocaux.⁵⁷ Permettre à l'athlète de dicter "valider série, 100 kilos, 8 répétitions, RPE 8" via une API de reconnaissance vocale permettrait un enregistrement totalement "mains libres", un avantage colossal lors des entraînements intenses.

Visualisation de Données : Le Choix Critique des Bibliothèques de Graphiques

Les graphiques illustrant la progression du volume d'entraînement, l'évolution du tonnage ou la distribution musculaire (sous forme de diagrammes circulaires) constituent le cœur visuel de l'application.¹ Le rendu de ces graphiques sur Android requiert une sélection minutieuse des bibliothèques de visualisation.

Historiquement, la communauté Android s'appuyait massivement sur la bibliothèque open

source MPAndroidChart.⁶³ Cependant, en 2026, cette bibliothèque est considérée comme technologiquement obsolète. Écrite intégralement en Java et conçue pour l'ancien système de vues impératif, elle souffre d'un manque de maintenance active, accumulant des milliers de problèmes non résolus sur son dépôt de code.⁶³ De plus, son architecture basée sur des primitives float engendre des problèmes de précision lors de la manipulation de grands nombres, tels que les horodatages Unix utilisés pour cartographier les entraînements sur de longues périodes.⁶³ Son intégration dans un projet moderne propulsé par Jetpack Compose s'avère particulièrement complexe et instable.⁶³

L'architecture moderne exige l'utilisation de bibliothèques natives pour Compose. Actuellement, la bibliothèque Vico est largement plébiscitée par l'industrie pour sa haute performance, sa flexibilité de personnalisation (marqueurs personnalisés, axes dynamiques) et sa parfaite intégration au paradigme réactif de Kotlin.⁶³ ComposeCharts constitue également une alternative viable et légère pour les diagrammes circulaires et les histogrammes basiques.⁶⁷ L'intégration de ces bibliothèques permet d'obtenir des visualisations fluides et interactives des métriques complexes de l'hypertrophie.

Intégrations Matérielles, Écosystème Android et Tâches Arrière-plan

Une application de fitness de classe mondiale ne vit pas en vase clos ; elle doit s'interfacer profondément avec le système d'exploitation et les périphériques portables (wearables) pour enrichir la capture des données et minimiser les interactions physiques avec le smartphone.

L'Interface de Programmation Health Connect

L'écosystème Android s'articule désormais autour de l'API Health Connect, remplaçant définitivement l'ancienne infrastructure Google Fit.⁶⁸ Intégrée nativement depuis Android 14 (et disponible en tant qu'application autonome sur les versions antérieures), Health Connect agit comme un référentiel centralisé et chiffré, permettant aux applications de partager des données biométriques de manière sécurisée sous le contrôle explicite de l'utilisateur.⁶⁹

L'application de musculation doit agir à la fois comme producteur (Write) et consommateur (Read) de données.⁷⁰

- Lorsqu'une séance est validée, l'application construit et injecte un objet `ExerciseSessionRecord` dans Health Connect.⁷² Cet enregistrement encapsule l'heure de début, l'heure de fin, et spécifie la nature de l'activité.
- En parallèle, l'application peut calculer l'énergie dépensée et insérer des objets `ActiveCaloriesBurned` (unités d'énergie sur un intervalle de temps donné).⁷³
- Du côté de la consommation, l'application peut extraire des séries temporelles de fréquence cardiaque (`HeartRate` - enregistrement de type *Series*) capturées par une montre connectée concurrente durant l'intervalle de la séance, permettant d'afficher l'intensité cardiovasculaire dans le résumé final de l'entraînement.⁵

D'un point de vue de l'ingénierie, l'interaction avec Health Connect requiert une gestion stricte

des permissions asynchrones.⁷² Pour préserver les performances de la batterie et éviter l'étranglement (throttling) des requêtes, les opérations d'écriture ne doivent pas être exécutées événement par événement. Elles doivent être regroupées en lots (Batching), l'API autorisant l'insertion de 1000 enregistrements simultanés par requête.⁷² Les zéros ne doivent être enregistrés que s'ils reflètent une véritable inactivité mesurée, et non une simple absence de port du périphérique.⁷⁴

Les Systèmes Embarqués : Wear OS et la Logique des Tuiles

La création d'une application compagnon pour les montres sous Wear OS est un vecteur de différenciation majeur, fortement plébiscité par les utilisateurs d'applications concurrentes comme Strong ou Hevy.⁹ L'objectif principal est de libérer l'athlète de la nécessité de conserver son téléphone sur lui durant l'exécution de ses exercices.⁹

L'architecture de l'application Wear OS doit s'appuyer sur le Material Design 3 Expressive, spécifiquement optimisé pour les écrans circulaires de petite taille.⁷⁶ L'interface s'articule autour du concept de "Tuiles" (Tiles), des widgets permettant un accès d'un simple glissement aux informations critiques.⁷⁶ Une disposition à trois emplacements (3-slot layout) est recommandée pour afficher, par exemple, le temps de repos restant, le nombre de répétitions cibles pour la prochaine série, et un bouton de validation immédiate.⁷⁷

La contrainte d'ingénierie centrale réside dans la gestion de la connectivité. Comme observé chez les concurrents performants, l'application Wear OS doit être capable de fonctionner de manière totalement autonome.⁹ Elle doit posséder sa propre instance de base de données Room pour enregistrer les séries lorsque la connexion Bluetooth avec le téléphone est rompue. La synchronisation bidirectionnelle doit s'opérer de manière opportuniste via un service dédié (CompanionDeviceService), garantissant une expérience fluide et sans perte de données.⁹

Chronomètres de Repos : Pièges Architecturaux des Tâches en Arrière-plan

L'une des fonctionnalités les plus critiques d'une application d'entraînement est le chronomètre de repos automatique se déclenchant à la fin d'une série.⁴³ Si cette implémentation semble triviale en apparence, elle représente un piège architectural redoutable dans le contexte du système d'exploitation Android, connu pour sa gestion agressive de la mémoire et de l'énergie (Doze Mode, App Standby).

Une erreur de conception fréquente consiste à utiliser l'API WorkManager pour programmer la notification d'alerte de fin de repos.⁴⁴ Bien que WorkManager soit l'outil standard pour la gestion asynchrone, il est architecturalement conçu pour des tâches "différables" dont l'exécution exacte n'est pas critique à la seconde près.⁴⁶ Par conception, WorkManager impose un intervalle minimum d'environ 15 minutes entre l'enregistrement d'une tâche et son exécution garantie.⁴⁴ Programmer un chronomètre de 90 secondes avec cet outil conduira invariablement à des déclenchements erratiques ou retardés, détruisant la fiabilité de l'application.

L'architecture requise pour un chronomètre de repos exige une exécution à l'instant exact ("Exact Alarms"). La solution technique impose l'utilisation de l'API AlarmManager (nécessitant

des permissions spécifiques depuis Android 12).⁴⁴ De plus, pour s'assurer que le système Android ne "tue" pas le processus de chronométrage pendant que l'utilisateur utilise une autre application (comme Spotify), le chronomètre doit idéalement être couplé à un service de premier plan ("Foreground Service"). Ce type de service affiche une notification persistante dans le tiroir du système, signalant à l'OS que l'application effectue un travail critique activement suivi par l'utilisateur, ce qui exempte le processus des politiques de restriction de la batterie.⁷⁹

Ingénierie Sociale et Boucles de Viralité

Bien qu'une application puisse se limiter à une excellence purement utilitaire et analytique (comme FitNotes), la domination du marché passe souvent par l'intégration de mécanismes de réseau social, brillamment exploités par des plateformes comme Hevy ou Strava.⁵ L'intégration de fonctionnalités communautaires transforme un simple journal en une plateforme de rétention puissante.

L'architecture du flux d'actualité ("Social Feed") nécessite une extension significative de la base de données vers le Cloud (généralement via des solutions Backend-as-a-Service comme Firebase Firestore ou Supabase).⁵ Le flux doit être capable d'agréger asynchronement les séances des profils suivis, affichant des résumés concis du tonnage, de la durée et des records personnels, soutenus par des interactions classiques de type mentions "J'aime" et commentaires.⁵

Cependant, la véritable innovation réside dans la friction réduite de l'échange de connaissances. Le système doit permettre le clonage de données. Lorsqu'un utilisateur navigue sur le profil d'un athlète et découvre une routine d'entraînement structurée, l'interface doit proposer un bouton d'importation direct ("Copy Routine").⁶ Sur le plan de la base de données, cette action duplique les enregistrements de la table Routine et Routine_Exercise du créateur vers l'identifiant (ID) de l'utilisateur cible, en ajustant ou en neutralisant les charges relatives pour prévenir tout risque de blessure.⁸

Pour alimenter la boucle de viralité d'acquisition de nouveaux utilisateurs, l'application doit générer des artefacts partageables. L'implémentation de composants de rendu permettant de convertir un programme d'entraînement en une image esthétiquement formatée ou en un lien hypertexte profond (Deep Link) permet aux utilisateurs de partager leurs méthodes sur des plateformes externes comme Reddit, WhatsApp ou X.⁵ Un utilisateur cliquant sur ce lien depuis un appareil mobile sans l'application installée sera redirigé vers le Play Store, et grâce à la persistance des paramètres de lien profond, se verra immédiatement proposer d'importer la routine spécifique une fois le processus d'installation achevé.⁸

Synthèse et Recommandations Opérationnelles

Le développement d'une application de musculation exhaustive sous Android dépasse largement le cadre du simple enregistrement de données textuelles. Il s'agit d'un exercice d'ingénierie complexe nécessitant l'harmonie entre une modélisation mathématique rigoureuse de la physiologie humaine, et une architecture logicielle hautement performante.

Pour pénétrer et s'imposer sur un marché dominé par des acteurs établis, le développement

doit impérativement s'aligner sur les trois piliers stratégiques suivants :

1. **Une intelligence analytique inébranlable** : L'intégration des ratios de volume pour les muscles primaires et secondaires, couplée au suivi de la proximité de l'échec (RPE/RIR), forme la base du moteur statistique. L'intégration algorithmique des repères de la Renaissance Periodization (MV, MEV, MAV, MRV) transformera l'application d'un journal passif en un outil de coaching prescriptif, répondant aux attentes des athlètes les plus exigeants en matière d'hypertrophie.
2. **Une ingénierie de la résilience** : Le choix de l'architecture "Offline-First" basée sur Room et les Coroutines Kotlin est non négociable pour garantir l'utilisabilité dans des environnements à connectivité restreinte. De même, l'adoption rigoureuse des API correctes pour les tâches d'arrière-plan (AlarmManager et Foreground Services pour les chronomètres, au lieu de WorkManager) déterminera la fiabilité fondamentale de l'outil aux yeux des utilisateurs finaux.
3. **Une ergonomie frictionless** : L'abandon des paradigmes de saisie textuelle classiques au profit de sélecteurs numériques (NumberPickers) natifs, associés à l'automatisation de la pré-saisie des charges historiques, réduira massivement le temps de friction. L'utilisation du Material Design 3 Expressive et de bibliothèques de rendu graphique modernes comme Vico assurera une lisibilité instantanée de ces vastes quantités de données générées.

En respectant ces spécifications architecturales et fonctionnelles, le produit final ne sera pas perçu comme un énième clone surchargé, mais comme un véritable écosystème analytique de référence, capable de capturer et de monétiser une audience dédiée à l'optimisation continue de la performance physique.

Sources des citations

1. Gym Performance Tracking (Tools, Graphs & Statistics) - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/gym-performance/>
2. 1 Workout Tracker & Planner Gym Log App - Hevy, consulté le avril 10, 2026, <https://www.hevyapp.com/use-cases/workout-log-app/>
3. Track Your Sets Per Muscle Group Per Week - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/sets-per-muscle-group-per-week/>
4. Fitbod vs Strong vs Hevy vs SensAI: 2026 Comparison, consulté le avril 10, 2026, <https://www.sensai.fit/blog/fitness-app-comparison>
5. Explore Hevy's Social Content Feed And Options - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/content-feed/>
6. Hevy's Social Features and How to Use Them - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/social-features/>
7. Learn How to Share Folders & Workout Routines - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/share-folders-routines/>
8. Hevy App Social Guide: Connect, Follow, and Share Your Workouts, consulté le avril 10, 2026, <https://help.hevyapp.com/hc/en-us/articles/35688036014231-Hevy-App-Social-Guide-Connect-Follow-and-Share-Your-Workouts>

9. Hevy vs Strong: Which Workout App Fits Your Training Style? - Setgraph, consulté le avril 10, 2026, <https://setgraph.app/ai-blog/hevy-vs-strong>
10. Strong vs Hevy Review – Still Good? - YouTube, consulté le avril 10, 2026, <https://www.youtube.com/watch?v=puhhAnkvfgc>
11. Why do you use Strong over Hevy (esp. now with the 6.0 update)? : r/strongapp - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/strongapp/comments/1jwqf1o/why_do_you_use_strong_over_hevy_esp_now_with_the/
12. FitNotes - Gym Workout Log - Apps on Google Play, consulté le avril 10, 2026, <https://play.google.com/store/apps/details?id=com.github.jamesgay.fitnotes>
13. FitNotes App Review (2025): A Minimalist, Ad-Free Workout Tracker Perfect for Indian Users, consulté le avril 10, 2026, https://www.reddit.com/r/Turbulencegains/comments/1k86x4n/fitnotes_app_review_2025_a_minimalist_adfree/
14. Best FitNotes Gym Workout Log app for Android - YouTube, consulté le avril 10, 2026, <https://www.youtube.com/watch?v=IVdlwfY4RRE>
15. How to count training volume and design a sensible training split - Menno Henselmans, consulté le avril 10, 2026, <https://mennohenselmans.com/how-to-count-training-volume-design-training-split/>
16. How Many Sets Per Muscle Group For Optimal Growth? - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/how-many-sets/>
17. My app is logging half sets and I don't of warm up sets. Anyone know why? : r/Hevy - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/Hevy/comments/1o8cw2j/my_app_is_logging_half_sets_and_i_dont_of_warm_up/
18. "Levels" Set Count : r/MacroFactor - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/MacroFactor/comments/1qfuk8c/levels_set_count/
19. Training Volume Landmarks for Muscle Growth - RP Strength, consulté le avril 10, 2026, <https://rpstrength.com/blogs/articles/training-volume-landmarks-muscle-growth>
20. Volume Landmarks - Renaissance Periodization, consulté le avril 10, 2026, <https://volume-landmarks-rp-rals.vercel.app/>
21. Summary of Dr. Mike Israetel and Renaissance Periodization's Hypertrophy Volume Landmarks : r/AdvancedFitness - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/AdvancedFitness/comments/lyney9/summary_of_dr_mike_israetel_and_renaissance/
22. Estimating the Volume Landmarks | Advanced Hypertrophy Concepts and Tools | Lecture 8, consulté le avril 10, 2026, <https://www.youtube.com/watch?v=M2PP8AAXgPo>
23. How to Activate RPE/RiR in Your Workouts - StrengthLog Support Area, consulté le avril 10, 2026, <https://help.strengthlog.com/help-article/how-to-activate-rpe-rir-in-your-workouts/>
24. A Guide To Using RPE and RIR In Your Training - Ripped Body®, consulté le avril 10, 2026, <https://rippedbody.com/rpe/>
25. Logging RPE and RIR with the Metric workout app, consulté le avril 10, 2026,

- <https://www.metric.coach/user-guide/rpe-and-rir>
26. How to Calculate and Log RPE During Training - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/how-to-calculate-rpe/>
 27. How to Use the Built-in Weight Plate Calculator - Hevy App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/weight-plate-calculator/>
 28. Stronglifts Plate Calculator + Setting Your Plates, consulté le avril 10, 2026, <https://support.stronglifts.com/article/154-plate-calculator>
 29. Using the Plate Calculator | MacroFactor, consulté le avril 10, 2026, <https://help.macrofactorapp.com/en/articles/313-using-the-plate-calculator>
 30. About Plate Calculator - Strong Help Center, consulté le avril 10, 2026, <https://help.strongapp.io/article/169-plate-calculator>
 31. App Launch - Plate Math: Easy barbell plate calculation : r/iOSProgramming - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/iOSProgramming/comments/1at7llv/app_launch_plate_math_easy_barbell_plate/
 32. Workout-Tracking Database - Brandon Carper, consulté le avril 10, 2026, <https://brandoncarper.com/workout-tracking-database>
 33. Is there a way to make an Android app run fully offline without losing important features?, consulté le avril 10, 2026, https://www.reddit.com/r/androiddev/comments/1oyk3fm/is_there_a_way_to_make_an_android_app_run_fully/
 34. Build an offline-first app | App architecture - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/topic/architecture/data-layer/offline-first>
 35. Guide to app architecture - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/topic/architecture>
 36. Data layer | App architecture - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/topic/architecture/data-layer>
 37. ♂FitTrack is a modern, privacy-focused fitness tracking app designed to help you track your runs, monitor your progress. - GitHub, consulté le avril 10, 2026, <https://github.com/akshatbhuhagal/FitTrack>
 38. allknowledge34/Fitness-Assest: Production-ready Health Tracker Android App built with Kotlin, Jetpack Compose, Firebase, Google Fit, and modern MVVM architecture featuring activity tracking, analytics, and offline-first system. - GitHub, consulté le avril 10, 2026, <https://github.com/allknowledge34/Fitness-Assest>
 39. Proper database design for a workout tracker : r/SQL - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/SQL/comments/tnin6m/proper_database_design_for_a_workout_tracker/
 40. Database design for workout tracking app. How do I model relation for exercise stats to a routine? [closed] - Stack Overflow, consulté le avril 10, 2026, <https://stackoverflow.com/questions/65974181/database-design-for-workout-tracking-app-how-do-i-model-relation-for-exercise-s>
 41. How to Build a Database Schema for a Fitness Tracking Application? - Tutorials - Back4app, consulté le avril 10, 2026, <https://www.back4app.com/tutorials/how-to-build-a-database-schema-for-a-fitness>

[-tracking-application](#)

42. Building a Modern Workout Tracker: A Full-Stack Journey | by Drake Damon | Medium, consulté le avril 10, 2026, <https://medium.com/@dddamon06/building-a-modern-workout-tracker-a-full-stack-journey-d9f404cd7b02>
43. Hevy App Feature List - #1 Workout Tracker & Gym Log App, consulté le avril 10, 2026, <https://www.hevyapp.com/features/>
44. Android Jetpack - WorkManager with Example - GeeksforGeeks, consulté le avril 10, 2026, <https://www.geeksforgeeks.org/kotlin/android-jetpack-workmanager-with-example/>
45. Mastering Background Tasks with WorkManager in Android | by Harsh Mittal - Medium, consulté le avril 10, 2026, <https://devharshmittal.medium.com/mastering-background-tasks-with-workmanager-in-android-4279d4d0f60a>
46. Task scheduling | Background work - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/develop/background-work/background-tasks/persistent>
47. A mobile app for complete fitness — UX case study | by Nisha Tibrewala - UX Collective, consulté le avril 10, 2026, <https://uxdesign.cc/fitterme-mobile-app-case-study-3d707104dce5>
48. Hi Guys! I am working on my very first UI/UX project (Fitness app) and have made quite a few screens and i would really appreciate your feedback. : r/FigmaDesign - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/FigmaDesign/comments/1jna70u/hi_guys_i_am_working_on_my_very_first_uiux/
49. Case Study: Hevy's New User Onboarding UX | by HSProdesign | Medium, consulté le avril 10, 2026, <https://uxplanet.org/new-user-onboarding-ux-hevys-activity-tracker-teardown-7b796b912636>
50. Fitness App UI - Android Compose Tutorial - YouTube, consulté le avril 10, 2026, https://www.youtube.com/watch?v=Bs_0ByUgAR0
51. Android Jetpack Compose - Build a BMI Calculator App from Scratch - GeeksforGeeks, consulté le avril 10, 2026, <https://www.geeksforgeeks.org/kotlin/android-jetpack-compose-build-a-bmi-calculator-app-from-scratch/>
52. How to make a BMI Calculator in Jetpack Compose - Medium, consulté le avril 10, 2026, <https://medium.com/@shivamkadam0709/how-to-make-a-bmi-calculator-in-jetpack-compose-7be33bf87f90>
53. Components — Material Design 3, consulté le avril 10, 2026, <https://m3.material.io/components>
54. Material Components | Mobile - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/design/ui/mobile/guides/components/material-overview>

55. Material Design 3 - Google's latest open source design system, consulté le avril 10, 2026, <https://m3.material.io/>
56. Fitness App UI Design: Key Principles for Engaging Workout Apps - Stormotion, consulté le avril 10, 2026, <https://stormotion.io/blog/fitness-app-ux/>
57. Onboarding Experience Design for Health and Fitness App — Get Fit | by Madhu Yadav, consulté le avril 10, 2026, <https://medium.com/@madhuyadav.mail02/onboarding-experience-design-for-health-and-fitness-app-get-fit-08c55aa09c1f>
58. Abdo-21/Jetpack-Compose-NumberPicker - GitHub, consulté le avril 10, 2026, <https://github.com/Abdo-21/Jetpack-Compose-NumberPicker>
59. NumberPicker | API reference - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/reference/android/widget/NumberPicker>
60. How can I make a number picker in jetpack compose? - Stack Overflow, consulté le avril 10, 2026, <https://stackoverflow.com/questions/75306878/how-can-i-make-a-number-picker-in-jetpack-compose>
61. FitNotes 2 - Gym Workout Log - App Store, consulté le avril 10, 2026, <https://apps.apple.com/se/app/fitnotes-2-gym-workout-log/id1538896016?l=en-GB>
62. Hevy - Workout Tracker Gym Log - App Store - Apple, consulté le avril 10, 2026, <https://apps.apple.com/us/app/hevy-workout-tracker-gym-log/id1458862350>
63. As of now, What is the best performed Android Charting Library? : r/androiddev - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/androiddev/comments/189mlv8/as_of_now_what_is_the_best_performed_android/
64. What are some good data visualization solutions and why? (Jetpack compose for ui) : r/androiddev - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/androiddev/comments/1ca2g26/what_are_some_good_data_visualization_solutions/
65. As of now, What is the best performed Android Charting opensource Library? - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/androiddev/comments/1o8ul3g/as_of_now_what_is_the_best_performed_android/
66. How to code Vico charts with Kotlin and Jetpack Compose. The best chart library so far!, consulté le avril 10, 2026, <https://www.youtube.com/watch?v=MY290aW8hMQ>
67. Best performance Compose Chart library : r/androiddev - Reddit, consulté le avril 10, 2026, https://www.reddit.com/r/androiddev/comments/1ky85q2/best_performance_compose_chart_library/
68. Health Connect | Android health & fitness - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/health-and-fitness/health-connect>
69. Health & fitness dev center - Android Developers, consulté le avril 10, 2026, <https://developer.android.com/health-and-fitness>
70. Get started with Health Connect | Android health & fitness - Android Developers, consulté le avril 10, 2026,

- <https://developer.android.com/health-and-fitness/health-connect/get-started>
71. Introducing Health Connect, a new API for Android app developers to securely access user health data, consulté le avril 10, 2026,
<https://android-developers.googleblog.com/2022/05/introducing-health-connect.html>
 72. Develop Workout Experiences with Health Connect | Android health & fitness, consulté le avril 10, 2026,
<https://developer.android.com/health-and-fitness/health-connect/experiences/workouts>
 73. Health Connect data types | Android health & fitness - Android Developers, consulté le avril 10, 2026,
<https://developer.android.com/health-and-fitness/health-connect/data-types>
 74. Write data | Android health & fitness, consulté le avril 10, 2026,
<https://developer.android.com/health-and-fitness/health-connect/write-data>
 75. Hevy users — how do you usually track your workouts? - Reddit, consulté le avril 10, 2026,
https://www.reddit.com/r/Hevy/comments/1oeeg52/hevy_users_how_do_you_usually_track_your_workouts/
 76. Wear | UI Design - Android Developers, consulté le avril 10, 2026,
<https://developer.android.com/design/ui/wear>
 77. Designing with personality: Introducing Material 3 Expressive for Wear OS, consulté le avril 10, 2026,
<https://android-developers.googleblog.com/2025/08/introducing-material-3-expressive-for-wear-os.html>
 78. Synchronize data | Android health & fitness, consulté le avril 10, 2026,
<https://developer.android.com/health-and-fitness/health-connect/sync-data>
 79. Background tasks overview | Background work - Android Developers, consulté le avril 10, 2026,
<https://developer.android.com/develop/background-work/background-tasks>
 80. android - How to reduce time of PeriodicWorkManager in WorkManager - Stack Overflow, consulté le avril 10, 2026,
<https://stackoverflow.com/questions/55135999/how-to-reduce-time-of-periodicworkmanager-in-workmanager>